Assign 1

과제 풀이 정리 · Notion 아카이브

5a÷문장 →Feature(특징)추출 - key = lambda x: (len(x),x)

|text| -> shortest word

==========

```
def find_shortest_lexicographically_first_word(text: str) -> str:
    return min(text.split(), key=lambda x: (len(x), x), default="")
```

==========

- text.split()은 문자열을 공백 기준으로 나눈 단어 리스트를 반환한다.
- key = lambda x: (len(x), x)는 다음 두 기준으로 비교한다.
  1. len(x): 길이가 짧은 순
  2. x: 길이가 같으면 사전 순서
- default=""는 text.split()의 결과가 빈 리스트일 때, 즉 text가 빈 문자열이거나 공백만 포함된 경우에 반환되는 기본값이다.

다중 조건 정렬

lambda x: (priority(x), x)

=> priority 기준으로 분류.

5b÷Euclidean Distance(유클리드 거리)구현

position1, position2 -> 유클리드 거리

이 함수는 2차원 좌표에만 적용된다. loc1과 loc2는 각각 (x, y) 형태의 2차원 위치이다.

==========

```
def euclidean_distance(loc1: Position, loc2: Position) -> float:
    return ((loc1[0] - loc2[0]) ** 2 + (loc1[1] - loc2[1]) ** 2) ** (1 / 2)
```

==========

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

5c÷문장 변형 - defaultdict(set)

문장 -> list of all "similar"

"similar"?

- same number of words
- pair of adjacent words

=> original에서 발생했던 인접 단어만 쓸 수 있음

- first word/last word 같음

EX>

Input : 'the red the blue the end'

Output : ['the red the blue the end',
'the blue the red the end',
'the red the red the end',
'the blue the blue the end']

==========

```
def mutate_sentences(sentence: str) -> List[str]:
    words: list[str] = sentence.split()
    if not words:
        return []
    graph: DefaultDict[str, Set[str]] = collections.defaultdict(set)
    for i in range(len(words) - 1):
        graph[words[i]].add(words[i + 1])
    target_length = len(words)
    end_word = words[-1]
    result: List[str] = []
    def dfs(current_path: List[str]) -> None:
        if len(current_path) == target_length:
            if current_path[-1] == end_word:
                result.append(" ".join(current_path))
            return
        current_word = current_path[-1]
        for next_word in graph[current_word]:
            current_path.append(next_word)
            dfs(current_path)
            current_path.pop()
    dfs([words[0]])
    return result
```

==========

5d÷ dot_product

v1, v2: Sparse Vector -> dot_product

Sparse vector는 값이 0인 원소를 저장하지 않는 표현 방식이다. 따라서 존재하지 않는 키의 값은 0으로 간주한다.

==========

```
def sparse_vector_dot_product(v1: SparseVector, v2: SparseVector) -> float:
```

```
    if len(v1) > len(v2):
        v1, v2 = v2, v1  # 이제 len(v1) <= len(v2)이다.
    return sum(v * v2.get(k, 0) for k, v in v1.items())
```

==========

dot product

=> 같은 key를 가진 것 끼리 곱하고

-> 곱한 결과를 합하면 됨.

```
sum(v * v2.get(k,0) for k,v in v1.items())
```

에서 v = v1.value

v2.get(k,0) = v2.value

get(k,0) : k에 해당하는 value를 가져오되,

없으면, 0을 반환

5e÷increment

|v1|, scale, |v2|

-> perform : v1 = v1 + scale * v2

v1 = v1 + scale * v2를 수행한다. v1은 weight, v2는 feature 벡터이다. SparseVector에서는 존재하지 않는 키의 값을 0으로 간주한다.

==========

```
def increment_sparse_vector(v1: SparseVector, scale: float, v2: SparseVector) -> None:
    if scale == 0:
        return
    for key, value in v2.items():
        v1[key] = v1.get(key, 0) + scale * value
```

5f ~~:~~most_frequent_words - defaultdict(int)

문장 -> most_frequent_words

==========

```
def find_most_frequent_words(text: str) -> Set[str]:
    if not text:
        return set()
    counts = collections.defaultdict(int)
    for word in text.split():
        counts[word] += 1
    if not counts:
        return set()
```

```
    max_value = max(counts.values())
    return {word for word, count in counts.items() if count == max_value}


이 함수는 최댓값과 동점인 모든 단어를 집합으로 반환한다.
==================================================
collections.defaultdict(int)
->디폴트 딕트를 만듦
(int) => 디폴트 값 = int()이 됨
=> int() -> 0 반환
```

← 인공지능 · 포스텍 목차로